CO5 AT2

1. **Design a pseudocode algorithm to perform data cleaning and transformation on time-series data. The algorithm should handle missing time stamps, interpolate missing values, remove outliers, and resample data at fixed intervals. Include steps for indexing by time, filtering invalid entries, and ensuring chronological consistency while maintaining efficiency.**

   Data Cleaning and Transformation of Time-Series Data

   Aim

   To design an efficient algorithm for cleaning and transforming time-series data by handling missing timestamps, missing values, outliers, and resampling the data at fixed time intervals.

   Pseudocode

   ALGORITHM Clean_TimeSeries(Data, Interval)

   INPUT:
   Data = time-series dataset containing Timestamp and Value
   Interval = required fixed sampling interval

   OUTPUT:
   Cleaned and transformed time-series dataset

   BEGIN

1. LOAD the time-series Data.

2. SELECT required columns:
    Timestamp, Value

3. CONVERT Timestamp column into DateTime format.

4. FILTER invalid entries:
    Remove rows where Timestamp is invalid.
    Remove rows where Value is invalid or non-numeric.

5. REMOVE duplicate timestamps:
    Keep the valid record for each timestamp.

6. SET Timestamp as the TIME INDEX.

7. SORT the data by Timestamp in ascending order.

8. CHECK chronological consistency:
    IF any timestamp is out of order
        Sort the data again by Timestamp.
    END IF

9. DETECT missing timestamps:
    Create a complete time range from
    minimum Timestamp to maximum Timestamp
    using the required Interval.

10. REINDEX the dataset using the complete time range.

11. HANDLE missing values:
    IF values are missing
        Use linear interpolation between valid values.
    END IF

12. DETECT OUTLIERS:
    Calculate statistical limits using IQR
    or another suitable outlier method.

    Lower Limit = Q1 - 1.5 × IQR
    Upper Limit = Q3 + 1.5 × IQR

13. REMOVE or REPLACE outliers:
    IF Value < Lower Limit OR Value > Upper Limit
        Mark the value as invalid.
        Replace it using interpolation or another
        suitable method.
    END IF

14. RESAMPLE the cleaned data:
    Resample the time-series at the fixed Interval.
    Calculate the mean/median value for each interval.

15. INTERPOLATE any remaining missing values
    after resampling.

16. VERIFY the final dataset:
    Check that:
      - timestamps are chronological
      - timestamps follow the fixed interval

- no invalid values remain
- missing values are handled

17. RETURN the cleaned and transformed dataset.

END

Flow of the Algorithm

Raw Time-Series Data

↓

Convert Timestamp to DateTime

↓

Filter Invalid Entries

↓

Remove Duplicate Timestamps

↓

Set Time Index

↓

Sort Chronologically

↓

Detect Missing Timestamps

↓

Reindex Complete Time Range

↓

Interpolate Missing Values

↓

Detect & Remove Outliers

↓

Resample at Fixed Interval

↓

Final Validation

↓

Clean Time-Series Data

Efficiency Considerations

- Use **time indexing** for fast time-based operations.
- Sort timestamps only when necessary.
- Use vectorized operations instead of processing each row individually.
- Use built-in interpolation and resampling functions.
- Avoid repeatedly scanning the entire dataset.
- This makes the algorithm suitable for **large time-series datasets**.

Conclusion

The algorithm produces a **chronologically consistent, complete, outlier-free, and uniformly sampled time-series dataset**. It improves data quality while maintaining computational efficiency for further analysis and machine-learning applications.

2. **Write a detailed pseudocode algorithm to implement feature selection on a dataset. The algorithm should compute correlation between features, remove redundant or irrelevant attributes, and retain significant features. Include steps for normalization, threshold-based filtering, and validation to ensure improved model performance and reduced dimensionality.**

Feature Selection Using Correlation and Threshold-Based Filtering

Aim

To design an efficient feature selection algorithm that identifies significant features, removes irrelevant and redundant attributes, reduces dimensionality, and improves machine-learning model performance.

Pseudocode

ALGORITHM Feature_Selection(Data, Target, Threshold)

INPUT:

Data = Dataset containing input features and target variable

Target = Target/output variable

Threshold = Correlation threshold

OUTPUT:

Selected_Features = Reduced dataset containing significant features

BEGIN

1. LOAD the dataset.

2. SEPARATE the dataset into:

X = Input features

Y = Target variable

3. CHECK the dataset for:

- Missing values
- Duplicate records
- Invalid or non-numeric values

4. HANDLE missing values:

Replace missing numerical values using mean/median

or remove records if necessary.

5. NORMALIZE the numerical features:

Scale each feature to a common range.

Example:

X_normalized = (X - Minimum) / (Maximum - Minimum)

6. COMPUTE the correlation matrix:

Calculate correlation between every pair of features.

7. IDENTIFY redundant features:

```
FOR each pair of features (Fi, Fj)
   IF |Correlation(Fi, Fj)| >= Threshold
      Mark one of the highly correlated features
      as redundant.
   END IF
END FOR
```

8. REMOVE redundant features:

Keep the feature that has:

- stronger correlation with the target, OR
- greater relevance to the prediction task.

9. COMPUTE feature-to-target correlation:

For each remaining feature Fi,

calculate:

Correlation(Fi, Target)

10. APPLY relevance threshold:
    IF |Correlation(Fi, Target)| < Relevance_Threshold
        Remove Fi because it is considered irrelevant.
    ELSE
        Keep Fi.
    END IF

11. CREATE the Selected_Features dataset
    using only the retained features.

12. VALIDATE feature selection:
    Split the dataset into:
        Training set
        Testing set

13. TRAIN a machine-learning model using:
    a) Original features
    b) Selected features

14. COMPARE model performance using:
    - Accuracy
    - Precision
    - Recall
    - F1-score
    - RMSE or MAE, depending on the problem

15. CHECK dimensionality reduction:
    Compare:
        Number of original features

Number of selected features

16. IF the selected-feature model gives equal or better performance with fewer features
    ACCEPT the selected feature set.
    ELSE
    Adjust the threshold and repeat the selection process.
    END IF

17. RETURN Selected_Features.

END

Correlation-Based Selection

Correlation measures how strongly two variables are related.

- **High positive correlation** → both features contain similar information.
- **High negative correlation** → features are strongly related in opposite directions.
- **Correlation near zero** → weak linear relationship.

For example, if:

Correlation(Age, Income) = 0.92

the two features are highly correlated. Keeping both may introduce redundancy, so one can be removed based on its relationship with the target.

Feature Selection Flow

Dataset

↓

Separate Features & Target

↓

Handle Missing/Invalid Data

↓

Normalize Features

↓

Calculate Correlation Matrix

↓

Remove Highly Correlated Features

↓

Calculate Feature–Target Correlation

↓

Apply Threshold

↓

Select Significant Features

↓

Train Model

↓

Validate Performance

↓

Final Reduced Dataset

Example

Suppose the original dataset contains **10 features**.

Feature	Correlation with Target	Decision
Age	0.72	Keep
Income	0.81	Keep
Salary	0.83	Keep
Experience	0.65	Keep
Height	0.08	Remove
Weight	0.12	Remove

Feature	Correlation with Target	Decision
ID	0.01	Remove
Location_Code	0.05	Remove
Spending	0.76	Keep
Credit_Score	0.79	Keep

If **Income and Salary** have a correlation of **0.95**, they are redundant. The feature with stronger relevance to the target can be retained.

Thus, the number of features may be reduced from **10 to 6**, while retaining the most useful information.

Efficiency and Validation

- Correlation matrices allow multiple features to be evaluated systematically.
- Normalization puts features on a comparable scale.
- Threshold-based filtering automatically removes weak or redundant features.
- Reduced features decrease **memory usage and computation time**.
- Validation ensures that dimensionality reduction does not reduce model performance.

Conclusion

The proposed feature-selection algorithm **normalizes the data, calculates feature correlations, removes redundant and irrelevant attributes using thresholds, and validates the selected features using model performance**. This reduces dimensionality, improves

computational efficiency, and can improve the accuracy and generalization of the machine-learning model.